2. 핵심 데이터 구조 분석

2.1 Hash 구조체 - 시스템의 핵심 식별자

2.1.1 기본 구조

`Hash` 는 iroh-blob 시스템의 가장 기본적인 데이터 구조로, BLAKE3 해시값을 래핑하는 32바이트 배열입니다.

```rust
pub struct Hash(blake3::Hash);
```

2.1.2 주요 특징

콘텐츠 주소 지정 (Content Addressing)

- 데이터의 내용이 곧 주소가 되는 시스템
- 동일한 데이터는 항상 같은 해시를 생성
- 데이터 무결성 자동 보장

```rust
let data = b"hello world";
let hash1 = Hash::new(data);
let hash2 = Hash::new(data);
assert_eq!(hash1, hash2); // 항상 같은 해시
```

빈 데이터 최적화

시스템에서 자주 사용되는 빈 데이터에 대해 상수로 미리 정의:

```
pub const EMPTY: Hash = Hash::from_bytes([
175, 19, 73, 185, 245, 249, 161, 166, 160, 64, 77, 234, 54, 220, 201, 73,
155, 203, 37, 201, 173, 193, 18, 183, 204, 154, 147, 202, 228, 31, 50, 98,
]);
```

2.1.3 직렬화 및 표현

16진수 표현

```
pub fn to_hex(&self) -> String {
self.0.to_hex().to_string()
}

// 짧은 표현 (디버깅용)
pub fn fmt_short(&self) -> ArrayString<10> {
let mut res = ArrayString::new();
data_encoding::HEXLOWER
.encode_write(&self.as_bytes()[..5], &mut res)
.unwrap();
res
}
```

다양한 직렬화 지원

- **바이너리**: postcard를 통한 효율적인 직렬화 (32바이트)
- **텍스트**: JSON에서 16진수 문자열로 표현 (66바이트)
- **데이터베이스**: redb를 통한 고정 크기 저장

2.1.4 성능 최적화

메모리 효율성

```
impl AsRef<[u8]> for Hash {
fn  as_ref(&self) -> &[u8] {
self.0.as_bytes()
}
}

impl Borrow<[u8]> for Hash {
fn  borrow(&self) -> &[u8] {
self.0.as_bytes()
}
}
```

정렬 및 비교

```
impl Ord for Hash {
fn  cmp(&self, other: &Self) -> std::cmp::Ordering {
self.0.as_bytes().cmp(other.0.as_bytes())
```

```
}

}
```

2.2 BlobFormat - 데이터 형식 구분

2.2.1 형식 정의

```rust
#[derive(Clone, Copy, PartialEq, Eq, Debug, Default)]

pub  enum BlobFormat {

#[default]

Raw, // 일반 데이터

HashSeq, // 해시 시퀀스 (컬렉션)

}
```

2.2.2 용도별 구분

Raw 형식

- 일반적인 바이너리 데이터
- 파일, 이미지, 문서 등
- 단일 블롭으로 처리

HashSeq 형식

- 다른 해시들의 시퀀스
- 디렉토리나 컬렉션 표현
- 32바이트 배수 크기 제한

```rust
impl BlobFormat {
pub const fn is_raw(&self) -> bool {
matches!(self, BlobFormat::Raw)
}

pub const fn is_hash_seq(&self) -> bool {
matches!(self, BlobFormat::HashSeq)
}
}
```

2.3 HashAndFormat - 해시와 형식의 조합

2.3.1 구조 정의

```rust
pub struct HashAndFormat {
pub hash: Hash,
pub format: BlobFormat,
}
```

2.3.2 생성 방법

```rust
impl HashAndFormat {
// 일반 데이터용
```

```
pub fn raw(hash: Hash) -> Self {
Self { hash, format: BlobFormat::Raw }
}

// 컬렉션용
pub fn hash_seq(hash: Hash) -> Self {
Self { hash, format: BlobFormat::HashSeq }
}
}
```

2.3.3 문자열 표현

```
impl fmt::Display for HashAndFormat {
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
let mut slice = [0u8; 65];
hex::encode_to_slice(self.hash.as_bytes(), &mut slice[1..]).unwrap();
match self.format {
BlobFormat::Raw => {
// 64자리 16진수
write!(f, "{}", std::str::from_utf8(&slice[1..]).unwrap())
}
BlobFormat::HashSeq => {
// 's' 접두사 + 64자리 16진수
```

```
slice[0] = b's';
write!(f, "{}", std::str::from_utf8(&slice).unwrap())
}
}
}
}
```

예시:

- Raw: `ea8f163db38682925e4491c5e58d4bb3506ef8c14eb78a86e908c5624a67200f`
- HashSeq: `sea8f163db38682925e4491c5e58d4bb3506ef8c14eb78a86e908c5624a67200f`

2.4 HashSeq - 해시 시퀀스 구조

2.4.1 기본 구조

```
pub struct HashSeq(Bytes);
```

HashSeq는 여러 해시를 연속적으로 저장하는 구조체로, 32바이트 배수 크기의 `Bytes` 객체를 내부에 보관합니다.

2.4.2 생성 및 검증

```
impl HashSeq {
pub fn new(bytes: Bytes) -> Option<Self> {
if bytes.len() % 32 == 0 {
```

```
Some(Self(bytes))
} else {
None
}
}
}
```

크기가 32의 배수가 아니면 생성을 거부하여 데이터 무결성을 보장합니다.

2.4.3 반복자 구현

```
impl HashSeq {
pub fn iter(&self) -> impl Iterator<Item = Hash> + '_ {
self.0.chunks_exact(32).map(|chunk| {
let hash: [u8; 32] = chunk.try_into().unwrap();
hash.into()
})
}
}
```

2.4.4 효율적인 접근

```
impl HashSeq {
pub fn get(&self, index: usize) -> Option<Hash> {
```

```
if index < self.len() {
let hash: [u8; 32] = self.0[index * 32..(index + 1) * 32]
.try_into().unwrap();
Some(hash.into())
} else {
None
}
}

pub fn pop_front(&mut self) -> Option<Hash> {
if self.is_empty() {
None
} else {
let hash = self.get(0).unwrap();
self.0 = self.0.slice(32..); // 제로 카피 슬라이싱
Some(hash)
}
}
}
```

2.5 BlobTicket - 블롭 접근 티켓

2.5.1 구조 정의

```
pub struct BlobTicket {
node: NodeAddr, // 제공자 노드 주소
format: BlobFormat, // 블롭 형식
hash: Hash, // 블롭 해시
}
```

2.5.2 티켓의 역할

BlobTicket은 특정 블롭에 접근하기 위한 모든 정보를 포함하는 자체 완결적인 구조체입니다:

1. **어디서** (NodeAddr): 데이터를 제공하는 노드
2. **무엇을** (Hash): 요청할 데이터의 해시
3. **어떻게** (BlobFormat): 데이터 해석 방법

2.5.3 직렬화 형식

```
enum TicketWireFormat {
Variant0(Variant0BlobTicket),
}

struct Variant0BlobTicket {
node: Variant0NodeAddr,
format: BlobFormat,
hash: Hash,
```

```
}
```

미래 확장성을 위해 버전 구분자를 포함한 구조로 설계되었습니다.

2.5.4 문자열 표현

```
impl fmt::Display for BlobTicket {
fn  fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
write!(f, "{}", Ticket::serialize(self))
}
}
```

티켓은 `blob` 접두사와 Base32 인코딩을 사용하여 사람이 읽을 수 있는 형태로 표현됩니다.

예시:

```
blobAAEAE58FF8833241AC82D6FF7611046ED67B5072D142C588D0063E942D9A75502B6AAAA...
```

2.6 ChunkRanges - 청크 범위 관리

2.6.1 타입 별칭

```
pub  type ChunkRanges = RangeSet2<ChunkNum>;
```

ChunkRanges는 `range_collections` 크레이트의 `RangeSet2` 를 사용하여 불연속적인 청크 범위들을 효율적으로 표현합니다.

2.6.2 확장 메서드

```rust
pub trait ChunkRangesExt {
fn last_chunk() -> Self;
fn chunk(offset: u64) -> Self;
fn bytes(ranges: impl RangeBounds<u64>) -> Self;
fn chunks(ranges: impl RangeBounds<u64>) -> Self;
fn offset(offset: u64) -> Self;
}
```

2.6.3 사용 예시

```rust
// 바이트 범위를 청크 범위로 변환
let chunk_ranges = ChunkRanges::bytes(0..1024);

// 특정 청크만 선택
let single_chunk = ChunkRanges::chunk(5);

// 마지막 청크 선택
let last = ChunkRanges::last_chunk();

// 복합 범위 (0-10 청크와 100-110 청크)
let complex = ChunkRanges::chunks(0..10) | ChunkRanges::chunks(100..110);
```

2.6.4 효율성 고려사항

메모리 최적화

- `RangeSet2` 는 최대 2개의 경계까지 할당 없이 저장
- 대부분의 범위 요청은 연속적이므로 효율적

연산 최적화

- 범위 합집합, 교집합, 차집합 연산 지원
- 불연속적인 범위들의 효율적인 표현

2.7 데이터 구조 간의 관계

2.7.1 계층 구조

```
BlobTicket
├── NodeAddr (어디서)
├── Hash (무엇을)
└── BlobFormat (어떻게)
  ├── Raw → 일반 데이터
  └── HashSeq → HashSeq 구조체
    └── Hash[] (해시 배열)
```

2.7.2 변환 관계

```
// Hash → HashAndFormat
let  haf: HashAndFormat = hash.into(); // 기본값은 Raw
```

```rust
// HashAndFormat → BlobTicket (NodeAddr 필요)
let ticket = BlobTicket::new(node_addr, haf.hash, haf.format);


// BlobTicket → HashAndFormat
let haf: HashAndFormat = ticket.into();


// Bytes → HashSeq (검증 포함)
let hash_seq = HashSeq::try_from(bytes)?;
```

2.7.3 사용 패턴

1. 단일 파일 요청

```rust
let hash = Hash::new(file_data);
let haf = HashAndFormat::raw(hash);
let ticket = BlobTicket::new(provider, hash, BlobFormat::Raw);
```

2. 컬렉션 요청

```rust
let hashes: Vec<Hash> = files.iter().map(|f| Hash::new(f)).collect();
let hash_seq = HashSeq::from_iter(hashes);
let collection_hash = Hash::new(hash_seq.into_inner());
let ticket = BlobTicket::new(provider, collection_hash, BlobFormat::HashSeq);
```

3. 부분 요청

```
let ranges = ChunkRanges::bytes(1024..2048) | ChunkRanges::bytes(4096..8192);
let request = GetRequest::blob_ranges(hash, ranges);
```

2.8 성능 및 메모리 고려사항

2.8.1 제로 카피 최적화

- `Bytes` 사용으로 참조 카운팅 기반 메모리 공유
- `HashSeq::pop_front()` 에서 슬라이싱을 통한 제로 카피
- 해시 비교 시 바이트 배열 직접 비교

2.8.2 직렬화 효율성

- postcard: 바이너리 형식으로 최소 크기
- redb: 데이터베이스 저장을 위한 고정 크기 형식
- JSON: 사람이 읽을 수 있는 형식 (개발/디버깅용)

2.8.3 메모리 사용량

| 구조체 | 메모리 크기 | 직렬화 크기 |

|--------|----------|----------|

| Hash | 32 bytes | 32 bytes |

| BlobFormat | 1 byte | 1 byte |

| HashAndFormat | 33 bytes | 33 bytes |

| HashSeq | 8 bytes + data | data size |

| BlobTicket | ~100 bytes | ~70 bytes |

이러한 핵심 데이터 구조들은 iroh-blob 시스템의 기반을 이루며, 효율적인 콘텐츠 주소 지정과 데이터 무결성 보장을 가능하게 합니다.